Project

«Λειτουργικά Συστήματα»

Φάση 1

Βασική δομή δεδομένων

Για τον scheduler της φάσης 1 δημιουργήσαμε ένα struct Task το οποίο αποθηκεύει τα βασικά χαρακτηριστικά μίας διεργασίας (όνομα, process id, χρόνος εισόδου στην ουρά και κατάσταση διεργασίας). Η κατάσταση διεργασίας μπορεί να είναι ένα από τα : NEW, RUNNING, STOPPED, EXITED.

Φροντίζουμε ώστε η κατάσταση να είναι πάντα ενημερωμένη.

```
typedef struct Task{
    char name[10];
    int pid;
    int arrival_time;
    char execution_state[8];
}Task;
```

Ουρά εκτέλεσης

Δημιουργήσαμε ένα linked list το οποίο αντιπροσωπεύει την ουρά με τις προς-εκτέλεση διεργασίες. Η λίστα έχει τις συναρτήσεις extract() και insert() οι οποίες χρησιμοποιούνται για να εξάγουμε και να εισάγουμε δεδομένα στην λίστα αντίστοιχα.

Η εισαγωγή γίνεται από το τέλος της λίστας ενώ η εξαγωγή από την αρχή της.

Το πεδίο arrival_time το οποίο αποθηκεύει τον χρόνο εισόδου ενός στοιχείου στην ουρά, αρχικοποιείται αυτόματα με την εισαγωγή δεδομένων στην λίστα αυτή:

```
    if(strcmp(task.execution_state,"NEW") == 0)
        newNode->key.arrival_time = get_wtime();
```

Η συνθήκη if υπάρχει για την Round-Robin περίπτωση όπου δεν θέλουμε να ξαναμετρήσουμε τον χρόνο εισαγωγής για τις “STOPPED” διαδικασίες αλλά και τις “IO” διεργασίες της επόμενης φάσης.

Μέτρηση χρόνου

Μετράμε τον χρόνο ορίζοντας την get_wtime() συνάρτηση η οποία εκμεταλλεύεται την gettimeofday() συνάρτηση κάνοντας include την <sys/time.h>. Παράλληλα η συνάρτηση printTime() απλά τυπώνει τον χρόνο εκτέλεσης.

Ροή του προγράμματος

Το πρόγραμμα αρχικά διαχειρίζεται τα inputs του χρήστη στο command line. Καταλαβαίνει ποια πολιτική θέλει ο χρήστης να υλοποιήσει (RR ή FCFS) και αν έχει εισάγει σωστά τα arguments. Αν όχι, τερματίζει με αντίστοιχο μήνυμα:

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler FCFSS reverse.txt
Wrong arguments.
```

Εισάγοντας λάθος πολιτική

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler FCFS 12 homogeneous.txt
Wrong arguments.
```

Εισάγοντας λάθος πλήθος arguments

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler FCFS homoGENIOUS.txt
Error in reading file.
```

Εισάγοντας λάθος αρχείο

Αποθηκεύει την δοσμένη πολιτική στην μεταβλητή alg καθώς και το δοσμένο κβάντο στην μεταβλήτη quantum. Έπειτα εκτελεί την συνάρτηση readFile() η οποία διαβάζει το δοσμένο αρχείο και αρχικοποιεί τις διεργασίες με τα αντίστοιχα ονόματά τους σε έναν βοηθητικό πίνακα tasks[]. Η readFile() χρησιμοποιεί την συνάρτηση strtok() για να απομονώσει την τελευταία λέξη από κάθε γραμμή του δοσμένου αρχείου, καθώς η λέξη αυτή αποτελεί το όνομα της διεργασίας. Έπειτα εισάγει με την insert() τις διεργασίες στην linked list ορίζοντας την κατάστασή τους ως “NEW”.

Αμέσως μετά, ξεκινάει το κύριο loop του προγράμματος το οποίο τερματίζει μόλις η ουρά εκτέλεσης αδειάσει. Το loop κάθε φορά ξεκινάει εξάγοντας την επόμενη προς εκτέλεση διεργασία από την ουρά εκτέλεσης και ορίζοντας το path του αρχείου που θα εκτελεστεί.

```
// Extraction from list
current_process = extract();


// Path for execl
strcpy(path,"/home/max/project2/work/");
strcat(path,current_process.name);
```

(!) Θα πρέπει να αλλάξετε το string στην strcpy() με το δικό σας path που περιέχει τα αρχεία που θέλετε να εκτελέσετε.

(!) Θα πρέπει το αρχείο txt να έχει και μία κενή γραμμή από κάτω

Έπειτα εκτελείται ο κώδικας για την πολιτική που έχει δώσει ο χρήστης.

First-Come First-Served

Στην πολιτική αυτή, οι διεργασίες εκτελούνται με βάση τον χρόνο άφιξής τους και συγκεκριμένα δίνεται σε κάθε διεργασία όσος χρόνος χρειάζεται για να τερματίσει. Μόλις μία τερματίσει, εκτελείται η επόμενη μέχρι να τελειώσει κ.ο.κ

Με την χρήση της fork() δημιουργούμε μία διεργασία-παιδί, η οποία με την exec() εκτελεί το αρχείο που βρίσκεται στο path που έχουμε προσδιορίσει. Παράλληλα η διεργασία-γονέας περιμένει με την κλήση της wait() να τερματίσει η διεργασία-παιδί. Μόλις το παιδί τερματίσει, ο γονέας τυπώνει τον χρόνο εκτέλεσης με την χρήση της get_wtime().

Αξίζει να αναφέρουμε πως η fork() επιστρέφει δύο φορές, μία στο παιδί με τιμή 0 και μία στον πατέρα με το pid του παιδιού. Από το γεγονός αυτό μπορούμε να καθορίσουμε τις κινήσεις της κάθε διεργασίας.

Παράδειγμα εκτέλεσης 1:

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler FCFS homogeneous.txt
Dequeue process with name work7
Executing work7
process 8430 begins
process 8430 ends
        Elapsed time: 5.879412
        pid: 8430

Dequeue process with name work7
Executing work7
process 8455 begins
process 8455 ends
        Elapsed time: 11.468532
        pid: 8455

Dequeue process with name work7
Executing work7
process 8492 begins
process 8492 ends
        Elapsed time: 17.101592
        pid: 8492

Dequeue process with name work7
Executing work7
process 8529 begins
process 8529 ends
        Elapsed time: 22.713432
        pid: 8529

Dequeue process with name work7
Executing work7
process 8566 begins
process 8566 ends
        Elapsed time: 28.344745
        pid: 8566
```

Παράδειγμα εκτέλεσης 2:

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler FCFS reverse.txt
Dequeue process with name work7
Executing work7
process 8952 begins
process 8952 ends
        Elapsed time: 6.413847
        pid: 8952

Dequeue process with name work6
Executing work6
process 8989 begins
process 8989 ends
        Elapsed time: 11.229858
        pid: 8989

Dequeue process with name work5
Executing work5
process 9014 begins
process 9014 ends
        Elapsed time: 15.245543
        pid: 9014

Dequeue process with name work4
Executing work4
process 9039 begins
process 9039 ends
        Elapsed time: 18.458916
        pid: 9039

Dequeue process with name work3
Executing work3
process 9064 begins
process 9064 ends
        Elapsed time: 20.866008
        pid: 9064

Dequeue process with name work2
Executing work2
process 9077 begins
process 9077 ends
        Elapsed time: 22.474041
        pid: 9077

Dequeue process with name work1
Executing work1
process 9090 begins
process 9090 ends
        Elapsed time: 23.280835
        pid: 9090
```

Round-Robin

Στην πολιτική αυτή, δίνεται σε κάθε διεργασία ένα σταθερό κβάντο χρόνου για να εκτελεστεί. Ο scheduler «κοιμάται» για όσο χρόνο είναι το δοσμένο κβάντο με την nanosleep(). Εάν δεν προλάβει η διεργασία να ολοκληρωθεί στον χρόνο αυτό, ο scheduler ξυπνάει και την σταματάει κάνοντας την κατάστασή της “STOPPED” και την επανατοποθετεί στην ουρά εκτέλεσης καλώντας την insert().

Καλούμε την fork() όπως και πριν για να δημιουργήσουμε διεργασίες αλλά αυτή την φορά φροντίζουμε να δημιουργούμε διεργασίες μόνο για αυτές που είναι σε κατάσταση “NEW” καθώς οι “STOPPED” έχουν ήδη δημιουργηθεί. Τις “STOPPED” διεργασίες απλά τις συνεχίζουμε.

```
if(strcmp(current_process.execution_state,"NEW") == 0)
{
    int pid = fork();
```

Αναλυτικότερα:

Όταν τελειώνει ο χρόνος που έχει δοθεί σε μία διεργασία ο scheduler στέλνει το σήμα SIGSTOP στην διεργασία αυτή για να σταματήσει την εκτέλεσή της. Ομοίως όταν εξάγεται μια διεργασία από την ουρά εκτέλεσης με κατάσταση “STOPPED”, ο scheduler στέλνει το σήμα SIGCONT για να συνεχίσει την εκτέλεσή της. Όταν μία διεργασία-παιδί τερματίζει, στέλνει στον γονέα (scheduler) το σήμα SIGCHLD το οποίο «πιάνει» ο sigchldHandler() και ορίζει την κατάσταση της διεργασίας ως “EXITED”. Ο scheduler ανάλογα με το αν η κατάσταση της διεργασίας είναι ή δεν είναι “EXITED” τυπώνει τον χρόνο εκτέλεσης ή επανεισάγει την διεργασία στην ουρά εκτέλεσης αντίστοιχα.

Πρόβλημα που αντιμετωπίσαμε:

Στην αρχή θεωρούσαμε πως το σήμα SIGCHLD στέλνεται μόνο όταν τερματίζει η διεργασία παιδί. Αργότερα συνειδητοποιήσαμε πως το σήμα αυτό στέλνεται και όταν ξαναρχίζει ή όταν σταματάει προσωρινά η διεργασία παιδί. Αυτό μας δημιουργούσε προβλήματα στην αρχή καθώς ενεργοποιούσε τον handler πρόωρα. Καταφέραμε να λύσουμε το πρόβλημα αυτό βάζοντας το flag SA_NOCLDSTOP στην sigaction(). Το flag αυτό αποτρέπει την αποστολή του SIGCHLD στο πατέρα κατά την επανεκκίνηση ή τον σταματημό μιας διεργασίας. Έτσι καταφέραμε να περιορίσουμε την αποστολή του SIGCHLD μόνο στην περίπτωση τερματισμού της διεργασίας παιδί.

Παράδειγμα εκτέλεσης 1:

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler RR 1000 homogeneous.txt
Dequeue process with name work7
Executing work7
process 11930 begins
Dequeue process with name work7
Executing work7
process 11931 begins
Dequeue process with name work7
Executing work7
process 11944 begins
Dequeue process with name work7
Executing work7
process 11945 begins
Dequeue process with name work7
Executing work7
process 11958 begins
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
Dequeue process with name work7
Executing work7
process 11930 ends
```

```
Process work7 finished
        Elapsed time: 26.305908
        pid: 11930

Dequeue process with name work7
Executing work7
process 11931 ends
Process work7 finished
        Elapsed time: 26.925606
        pid: 11931

Dequeue process with name work7
Executing work7
process 11944 ends
Process work7 finished
        Elapsed time: 27.549676
        pid: 11944

Dequeue process with name work7
Executing work7
process 11945 ends
Process work7 finished
        Elapsed time: 28.177864
        pid: 11945

Dequeue process with name work7
Executing work7
process 11958 ends
Process work7 finished
        Elapsed time: 28.811238
        pid: 11958
```

Παράδειγμα εκτέλεσης 2:

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler RR 1000 reverse.txt
Dequeue process with name work7
Executing work7
process 13016 begins
Dequeue process with name work6
Executing work6
process 13017 begins
Dequeue process with name work5
Executing work5
process 13030 begins
Dequeue process with name work4
Executing work4
process 13031 begins
Dequeue process with name work3
Executing work3
process 13044 begins
Dequeue process with name work2
Executing work2
process 13045 begins
Dequeue process with name work1
Executing work1
process 13058 begins
process 13058 ends
Process work1 finished
        Elapsed time: 7.223143
        pid: 13058

Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
Dequeue process with name work5
Executing work5
Dequeue process with name work4
Executing work4
Dequeue process with name work3
Executing work3
Dequeue process with name work2
Executing work2
process 13045 ends
Process work2 finished
        Elapsed time: 12.835238
        pid: 13045

Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
Dequeue process with name work5
Executing work5
Dequeue process with name work4
Executing work4
Dequeue process with name work3
Executing work3
process 13044 ends
Process work3 finished
        Elapsed time: 17.245385
        pid: 13044
```

```
Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
Dequeue process with name work5
Executing work5
Dequeue process with name work4
Executing work4
process 13031 ends
Process work4 finished
        Elapsed time: 20.463196
        pid: 13031

Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
process 13017 ends
Process work6 finished
        Elapsed time: 22.284421
        pid: 13017

Dequeue process with name work5
Executing work5
process 13030 ends
Process work5 finished
        Elapsed time: 22.314753
        pid: 13030

Dequeue process with name work7
Executing work7
process 13016 ends
Process work7 finished
        Elapsed time: 22.938543
        pid: 13016
```

Φάση 2

Βασική δομή δεδομένων

Για την φάση 2 κρατήσαμε την ίδια δομή δεδομένων για να περιγράφουμε τις διεργασίες απλά θεωρήσαμε πως το πεδίο execution_state θα μπορεί να λαμβάνει και μία νέα τιμή/κατάσταση, την “IO”.

Ουρά εκτέλεσης

Σχετικά με την ουρά εκτέλεσης, επιλέξαμε να δημιουργήσουμε και μια ακόμα ουρά στην οποία θα δίνεται προτεραιότητα δηλαδή ο scheduler θα προτιμάει να εξάγει διεργασίες από την priority queue εάν αυτή περιέχει διεργασίες:

```
// Extraction from list
if(prio_first==NULL)
    {current_process = extract(&first);}
else
    {current_process = extract(&prio_first);}
```

Ο first είναι δείκτης στην αρχή της απλής λίστας και ο prio_first δείκτης στην αρχή της λίστας προτεραιότητας.

Για αυτό το λόγο χρειάστηκε να τροποποιήσουμε τις συναρτήσεις insert() και extract() ώστε να μπορούμε να προσδιορίσουμε σε ποια από τις λίστες θέλουμε να κάνουν εισαγωγή και εξαγωγή αντίστοιχα.

Στην λίστα προτεραιότητας έχουμε φροντίσει να εισάγονται οι διεργασίες που εκτελούν κάποια πράξη εισόδου/εξόδου. Οι διεργασίες αυτές εισάγονται στην λίστα προτεραιότητας αφού τελειώσουν το I/O.

Ροή του προγράμματος

Η γενική δομή του προγράμματος δεν έχει αλλάξει σε σχέση με την φάση 1. Με τον ίδιο τρόπο σκανάρουμε τα ορίσματα του χρήστη στο command line και με τον ίδιο τρόπο διαβάζουμε το αρχείο και αρχικοποιούμε την λίστα αναμονής. Η μόνη διαφορά είναι πως το κύριο loop τερματίζει μόνο όταν και οι δύο λίστες είναι άδειες και πως φροντίζουμε για την περίπτωση όπου οι λίστες αδειάσουν αλλά μία διεργασία πραγματοποιεί ακόμα I/O :

```
while ((first != NULL) || (prio_first !=NULL || io_state))
{
    if(io_state==1 && (first == NULL) && (prio_first == NULL)){continue;}
    else
```

Διαχείριση των “IO” διεργασιών

FCFS

Αρχικά, χρειάστηκε να περιορίσουμε τα fork() όπως κάναμε και στην φάση 1 για την πολιτική RR:

```
if(strcmp(current_process.execution_state,"NEW") == 0)
{
    int pid = fork();
```

Αυτό το κάναμε καθώς πλέον μπορεί να γίνει extracted από την λίστα προτεραιότητας μια διεργασία με κατάσταση “IO” για την οποία είχαμε ήδη κάνει fork() όταν εξήλθε από την βασική ουρά αναμονής με execution_state “NEW”. Στην φάση 1 δεν υπήρχε αυτή η ανάγκη για την FCFS καθώς κάθε διεργασία ερχόταν με κατάσταση “NEW” και έπειτα τερμάτιζε ενώ τώρα μία διεργασία που κάνει I/O αφαιρείται από την CPU για να μπει η επόμενη και επανεισάγεται όταν τελειώσει το I/O.

Σε αυτό το σημείο πρέπει να πούμε κάποια λόγια για την συνάρτηση wait() που χρησιμοποιούμε στην υλοποίηση της FCFS του extended scheduler.

Η wait() πρόκειται για μια συνάρτηση που επιστρέφει -1 σε περίπτωση που διακοπεί από ένα σήμα το οποίο «πιάνει» κάποιος signal handler. Το γεγονός αυτό, έχει ένα θετικό και ένα αρνητικό.

Θετικό

Στην περίπτωση που η επεξεργαζόμενη διεργασία πρόκειται να εκτελέσει I/O, στέλνει το σήμα SIGUSR1 στον scheduler. Το σήμα αυτό διακόπτει την wait() και άρα ο γονέας ξυπνάει πρόωρα εξάγοντας την επόμενη διεργασία. Αυτό είναι κάτι επιθυμητό καθώς θέλουμε να συνεχίσουμε να εκτελούμε διεργασίες για όση ώρα εκτελείται I/O.

Αρνητικό

Στην περίπτωση όμως όπου τελειώνει το I/O η διεργασία, στέλνεται το σήμα SIGUSR2 στον scheduler διακόπτοντας πάλι την wait(). Στην συγκεκριμένη όμως περίπτωση δεν θέλουμε να διακοπεί καθώς επιθυμούμε να τελειώσει πρώτα η διεργασία που εκτελείται τώρα (πρόκειται για FCFS).

Το πρόβλημα αυτό το αντιμετωπίζουμε βάζοντας το flag SA_RESTART στην sigaction() του SIGUSR2. Με αυτό το flag συνεχίζει η wait() την εκτέλεσή της. Παράλληλα, ο handler του SIGUSR2 εισάγει την διεργασία στην ουρά προτεραιότητας ώστε αφού τελειώσει η τωρινή διεργασία, να προτιμηθεί μετά αυτή που τελείωσε το I/O.

```
ioSig2.sa_handler = sigIOHandler2;
sigemptyset(&ioSig2.sa_mask);
ioSig2.sa_flags = SA_RESTART;
```

```
void sigIOHandler2(int signum) //SIGUSR2
{
    printf("Inserted to priority list\n");
    insert(io_process, &prio_first, &prio_last);
    io_state=0;
}
```

Έτσι, με το return value της wait() ανιχνέυουμε την περίπτωση όπου διακόπτεται η wait() και λαμβάνουμε τα απαραίτητα μέτρα:

```
else if(pid>0)              // Goneas
{
    current_process.pid = pid;
    if(wait(NULL)==-1) // elegxoyme an diakopei apo shma
    {
        strcpy(current_process.execution_state, "IO");
        io_process=current_process;
    }
    else
    {
        t1=get_wtime();
        printf("    Elapsed time: %lf\n", t1 - current_process.arrival_time);
        printf("    pid: %d\n\n",current_process.pid);
    }
```

Αν διακόπηκε, ορίζουμε την κατάστασή της “IO” και την αποθηκεύουμε σε μια μεταβλητή ώστε να την εισάγουμε στην ουρά προτεραιότητας αφού τελειώσει το I/O. Αν δεν διακόπηκε τότε απλά τυπώνουμε τον χρόνο εκτέλεσης. Με την συγκεκριμένη υλοποίηση θεωρούμε πως μόνο μία διεργασία μπορεί να κάνει I/O, αλλά μπορούμε εύκολα να το επεκτείνουμε και για πολλές κάνοντας την μεταβλητή io_process πίνακα (io_process[]) ή λίστα.

Αξίζει να αναφέρουμε πως η wait() μπορεί να επιστρέψει -1 και για άλλους λόγους όπως στην περίπτωση ανυπαρξίας παιδιού κ.α. Επομένως δεν έχουμε κάνει καλό error handling σε αυτή την υλοποίηση. Θα μπορούσαμε να ελέγξουμε το αν διακόπηκε η wait() συγκεκριμένα από ένα σήμα μέσα από την μεταβλητή errno που τίθεται από διάφορα system calls . Όμως θεωρήσαμε πως αυτό ξεφεύγει από τον σκοπό της άσκησης και για λόγους απλότητας και ευαναγνωσημότητας του κώδικα, κρατήσαμε την συνθήκη if(wait(NULL)==-1

<u>Παράδειγμα εκτέλεσης FCFS:</u>

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler_io FCFS mixed.txt
Dequeue process with name work5x2_io
Executing work5x2_io
process 8740 begins
process 8740 starts io
Dequeue process with name work7
Executing work7
process 8753 begins
Inserted to priority list
process 8753 ends
        Elapsed time: 10.207154
        pid: 8753

Dequeue process with name work5x2_io
Executing work5x2_io
process 8740 completed io
process 8740 ends
        Elapsed time: 14.214133
        pid: 8740

Dequeue process with name work6
Executing work6
process 8778 begins
process 8778 ends
        Elapsed time: 18.996911
        pid: 8778
```

RR

Αρχικά προσπαθήσαμε να εφαρμόσουμε την ίδια προσέγγιση και για την πολιτική Round-Robin. Όμως, η nanosleep() που χρησιμοποιούμε αντί για την wait() ώστε να «κοιμηθεί» ο scheduler για το δοσμένο κβάντο χρόνου, δεν συνεχίζει την εκτέλεσή της όταν διακοπεί από οποιοδήποτε από τα SIGUSR1 και SIGUSR2, ακόμα και με το flag SA_RESTART.

Όπως και πριν, δεν αποτελεί πρόβλημα η διακοπή που προέρχεται από το SIGUSR1 αλλά η διακοπή από το SIGUSR2. Για να ξεχωρίσουμε την περίπτωση που θέλουμε να διακοπεί από την περίπτωση που δεν θέλουμε, χρησιμοποιήσαμε μια μεταβλητή io_state που μας δείχνει αν κάποια διεργασία πραγματοποιεί I/O. Για αυτό, ο handler του SIGUSR1 θέτει την μεταβλητή 1 και ο handler του SIGUSR2 την κατεβάζει σε 0.

Το SIGUSR1 και το SIGUSR2 διακόπτουν πάντα την nanosleep() η οποία επιστρέφει -1 και αποθηκεύει τον εναπομείναντα χρόνο στην μεταβλητή remaining. Στην περίπτωση που διακόπτεται από το SIGUSR2 θέλουμε να ξανακαλέσουμε την nanosleep() για τον χρόνο που απομένει:

```
if(nanosleep(&request, &remaining)==-1 && (io_state==1)&& strcmp(current_process.execution_state,"EXITED")!=0)   // an termatisei apo signal
{
    strcpy(current_process.execution_state, "IO");
    io_process=current_process;
}
else
{
    if(io_state==0 && strcmp(current_process.execution_state,"EXITED") != 0)
        nanosleep(&remaining, &remaining);
```

Όμως η nanosleep() επιστρέφει -1 και όταν μια διεργασία τερματίζει από μόνη της επειδή τελείωσε (σήμα SIGCHLD). Σε αυτή την περίπτωση δεν θέλουμε να την ξανακαλέσουμε (καθώς θα μπορούσαμε να ξεκινήσουμε την επόμενη, κερδίζοντας χρόνο) και για αυτό ελέγχουμε πρώτα αν είναι “EXITED” η current process.

Παράδειγμα εκτέλεσης RR:

```
max@DESKTOP-548569G:~/project2/scheduler$ ./scheduler_io RR 1000 mixed.txt
Dequeue process with name work5x2_io
Executing work5x2_io
process 9453 begins
Dequeue process with name work7
Executing work7
process 9454 begins
Dequeue process with name work6
Executing work6
process 9461 begins
Dequeue process with name work5x2_io
Executing work5x2_io
Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
Dequeue process with name work5x2_io
Executing work5x2_io
Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
Dequeue process with name work5x2_io
Executing work5x2_io
process 9453 starts io
Dequeue process with name work7
Executing work7
Dequeue process with name work6
Executing work6
Dequeue process with name work7
Executing work7
Inserted to priority list
Dequeue process with name work5x2_io
Executing work5x2_io
process 9453 completed io
Dequeue process with name work6
Executing work6
process 9461 ends
Process work6 finished
        Elapsed time: 15.259847
        pid: 9461

Dequeue process with name work7
Executing work7
process 9454 ends
Process work7 finished
        Elapsed time: 15.835806
        pid: 9454

Dequeue process with name work5x2_io
Executing work5x2_io
Dequeue process with name work5x2_io
Executing work5x2_io
Dequeue process with name work5x2_io
Executing work5x2_io
process 9453 ends
Process work5x2_io finished
        Elapsed time: 18.847273
        pid: 9453

max@DESKTOP-548569G:~/project2/scheduler$
```